

Software Requirement Specification (SRS)

Project Name: SkillShurokkha - AI Verified Freelance Marketplace with Secure Payment

Date: June 05, 2026

Version: v7.0

By: Group SkillShurokkha

Members: Md Monjurul Islam (2039), Maisha Khatun (2085), Md Rubel Mia (2163)

Revision history			
Version	Author	Version Description	Date Completed
V1.0	Md Monjurul Islam (2039) Maisha Khatun (2085) Md Rubel Mia (2163)	1st Version of SRS based on project idea and scope	04-01-2026
V2.0	Md Monjurul Islam (2039) Maisha Khatun (2085) Md Rubel Mia (2163)	2nd Version of SRS based on user roles and core features	18-01-2026
V3.0	Md Monjurul Islam (2039) Maisha Khatun (2085) Md Rubel Mia (2163)	3rd Version of SRS based on project management and escrow flow	01-02-2026
V4.0	Md Monjurul Islam (2039) Maisha Khatun (2085) Md Rubel Mia (2163)	4th Version of SRS based on AI skill verification workflow	08-02-2026
V5.0	Md Monjurul Islam (2039) Maisha Khatun (2085) Md Rubel Mia (2163)	5th Version of SRS based on UI design and dashboard screens	22-02-2026

V6.0	Md Monjurul Islam (2039) Maisha Khatun (2085) Md Rubel Mia (2163)	6th Version of SRS based on UML and system design diagrams	01-03-2026
V7.0	Md Monjurul Islam (2039) Maisha Khatun (2085) Md Rubel Mia (2163)	7th Version of SRS based on final SkillShurokkha project	05-06-2026

Review History

Approving Party	Version Approved	Signature	Date
Course Teacher / Supervisor	v1.0		

Approval History

Reviewer	Version Reviewed	Signature	Date
Project Evaluator	v1.0		

Table of Contents

1. Introduction	01
1.1 Product Scope	01
1.2 Product Value	02
1.3 Intended Audience	02
1.4 Intended Use	02
1.5 General Description	03
2. Functional Requirements	04
2.1 Design Requirements	04
2.2 Graphics Requirements	04
2.3 Operating System Requirements	05
2.4 User Interface Design	05
2.5 UML and System Design	18
2.6 Constraints	24
3. External Interface Requirements	25
4. Non-functional Requirements	27
5. Definitions and Acronyms	29

1. Introduction

This Software Requirement Specification document describes the requirements of SkillShurokkha, an AI verified freelance marketplace with secure payment support. The system is designed to reduce trust problems in freelancing by verifying freelancer skills, managing client projects, protecting payment through escrow, and supporting communication between clients, freelancers, and administrators.

1.1 Product Scope

1.1.1 Benefits

- Helps freelancers prove their real skills through video based verification.
- Helps clients hire trustworthy and skilled freelancers.
- Reduces payment fraud by locking payment in escrow before work starts.
- Provides project, messaging, notification, and dispute handling in one platform.
- Supports Bangla and English interface for better local usability.

1.1.2 Objectives

- To provide role based accounts for freelancers, clients, and administrators.
- To allow freelancers to submit skill verification videos and receive AI scores.
- To allow clients to create projects, review applicants, fund escrow, and hire freelancers.
- To provide secure work submission, revision, approval, and payment release workflows.
- To notify users about important project, payment, message, and verification events.

1.1.3 Goals

- To build a trusted Bangladeshi freelance marketplace.
- To improve transparency between clients and freelancers.
- To provide a modern web platform that is easy to use on desktop and mobile devices.

1.2 Product Value

SkillShurokkha creates value by solving two major freelancing problems: fake skill claims and insecure payment. Freelancers can build credibility through verified badges and work history. Clients can select applicants with more confidence. Escrow payment protects both parties because the client funds the project before hiring, while the freelancer receives payment only after approved work submission.

1.3 Intended Audience

- Freelancers such as designers, developers, writers, editors, and digital service providers.
- Clients, small businesses, startups, and agencies looking for verified talent.

- Platform administrators responsible for verification, payment review, and dispute handling.
- Teachers, supervisors, and evaluators who will review the software project.

1.4 Intended Use

Freelancers use SkillShurokkha to register, complete their profile, apply to projects, submit skill verification videos, communicate with clients, submit work, and receive payments. Clients use the system to post projects, review applications, shortlist applicants, fund escrow, hire freelancers, review submitted work, request revisions, and approve payment release. Administrators use the system to monitor users, review skill verifications, confirm manual payments, and resolve disputes.

1.5 General Description

SkillShurokkha is a web based freelance marketplace application. The application includes authentication, role based dashboards, project management, proposal submission, AI assisted skill verification, escrow payment, real time messaging, notifications, and administrator review features. The frontend is built with Next.js and React. The backend is built with Node.js and Express. Data is stored in a relational database, and payment providers such as bKash, Nagad, bank transfer, or development gateway can be integrated through payment provider modules.

2. Functional Requirements

2.1 Design Requirements

- The system shall have separate dashboard experiences for admin, freelancer, and client users.
- The system shall use a sidebar based dashboard layout for quick navigation.
- The interface shall support both Bangla and English language modes.
- The system shall use responsive layouts so that pages work on desktop and mobile screens.
- The design shall use clear visual states for active navigation, unread notifications, project status, payment status, and verification status.

2.2 Graphics Requirements

- The system shall use icons for navigation, project actions, payment, message, notification, verification, and settings controls.
- The dashboard shall use cards to show statistics such as active projects, profile completion, payment summary, and verification status.
- The project and applicant pages shall use visual cards and badges for status readability.
- The application shall use a consistent green, mint, cyan, and white visual theme.
- All UI graphics shall load quickly and remain readable on low bandwidth devices.

2.3 Operating System Requirements

- The web application shall run on Windows, Linux, Android, and iOS devices through a modern browser.
- Supported browsers shall include Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari.
- Camera and microphone access shall be required only when recording or uploading skill verification media.
- An internet connection shall be required for authentication, project updates, messaging, payment, and notifications.

2.4 User Interface Design

The following figures represent the Figma style user interface design of the SkillShurokkha web application. These screens are based on the main workflows implemented in the project.

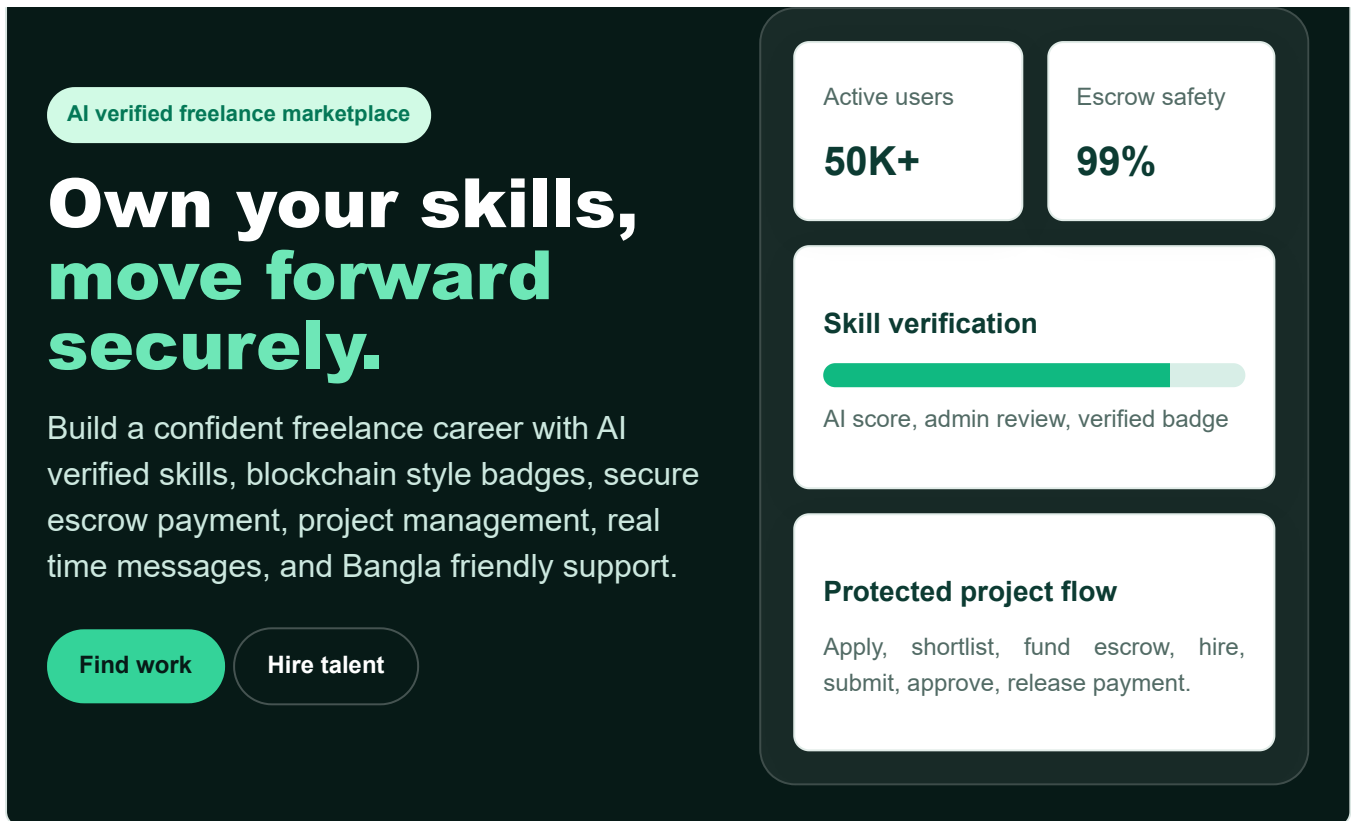


Fig-2.4.1: Landing page

The Landing Page is the public first screen of SkillShurokkha. It presents the platform brand, language switch, sign in/get started actions, and main value proposition: AI verified skills, trusted hiring, and secure escrow payment. The design follows the actual project theme with dark green background, mint action buttons, and trust focused dashboard preview cards.

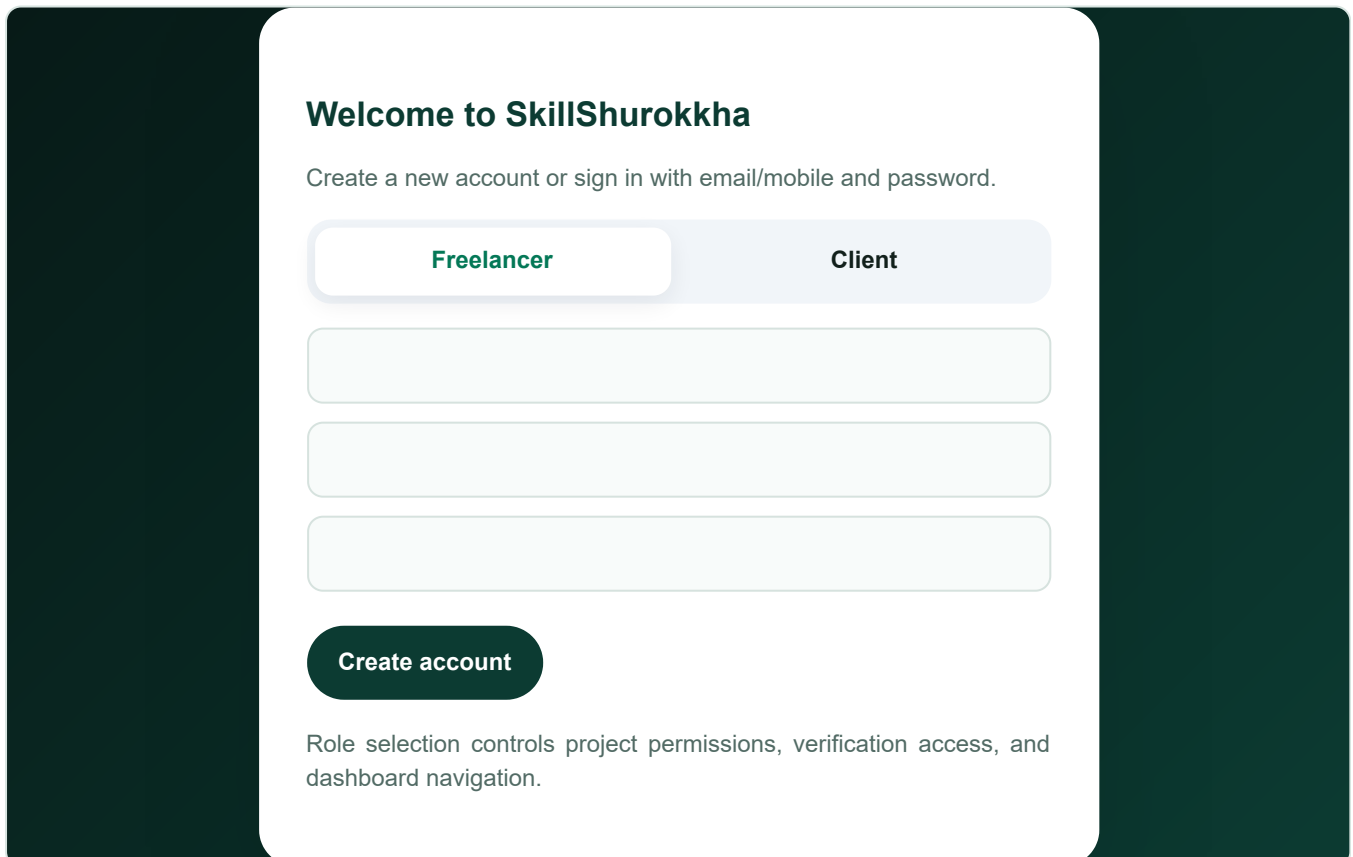


Fig-2.4.2: Authentication and role selection modal

The Authentication Modal is used for registration and login in the current web project. During registration, users select either Freelancer or Client role. This selected role determines access to project application, project posting, payment, skill verification, and dashboard features.

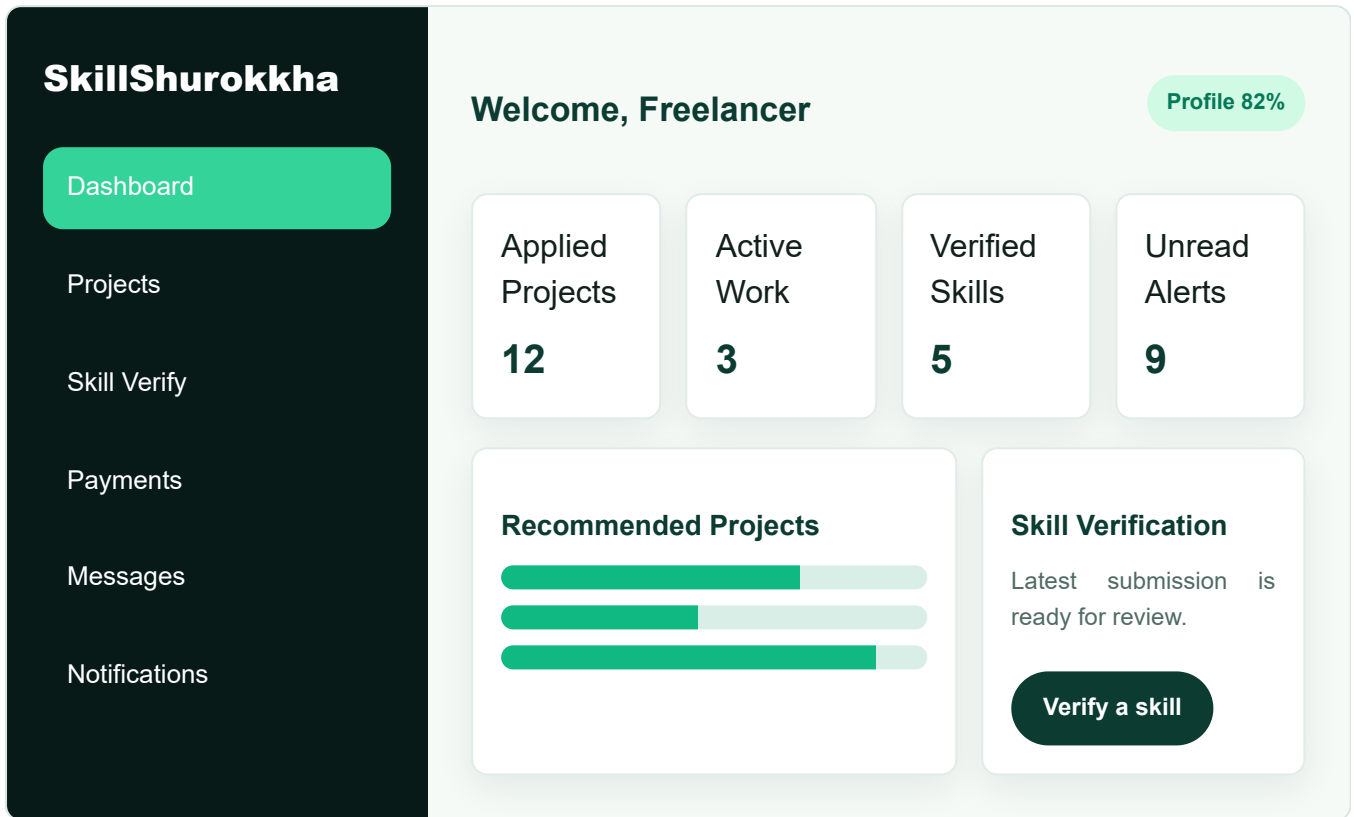
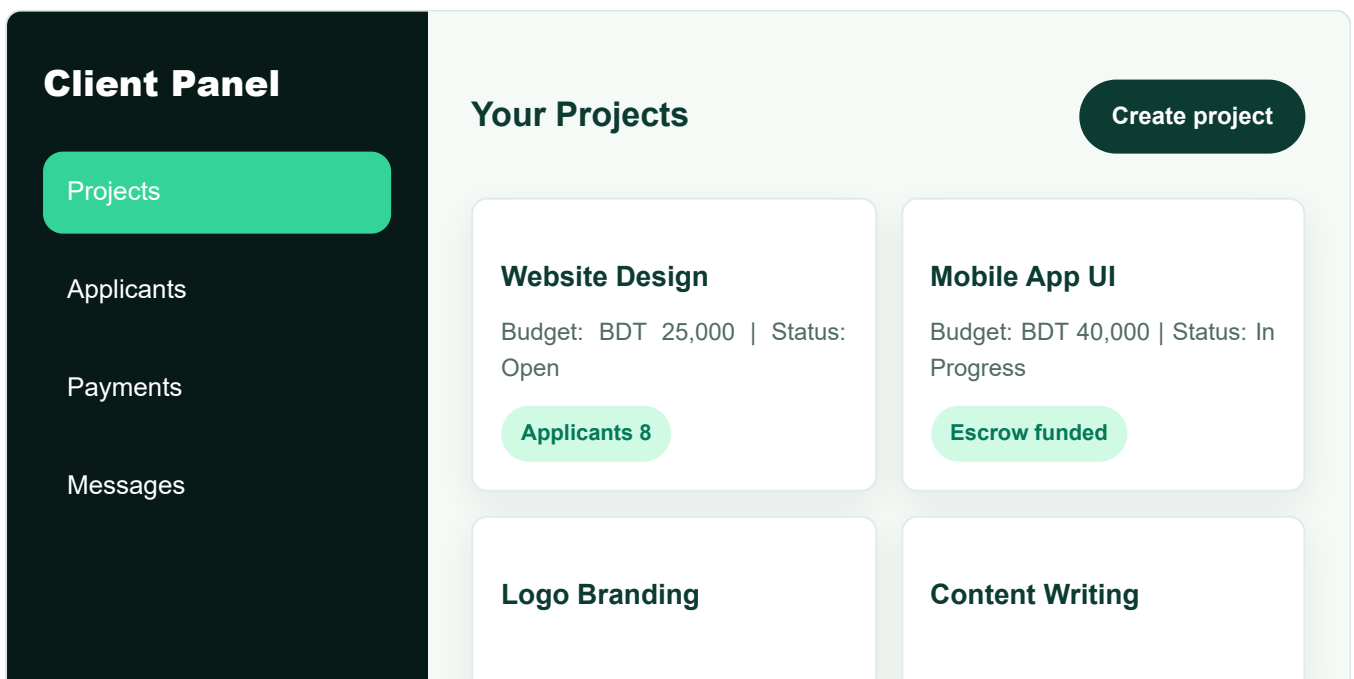


Fig-2.4.3: Freelancer dashboard

The Freelancer Dashboard displays profile completion, applied projects, active work, verified skills, latest verification status, and recommended projects. It provides quick access to skill verification and project browsing.



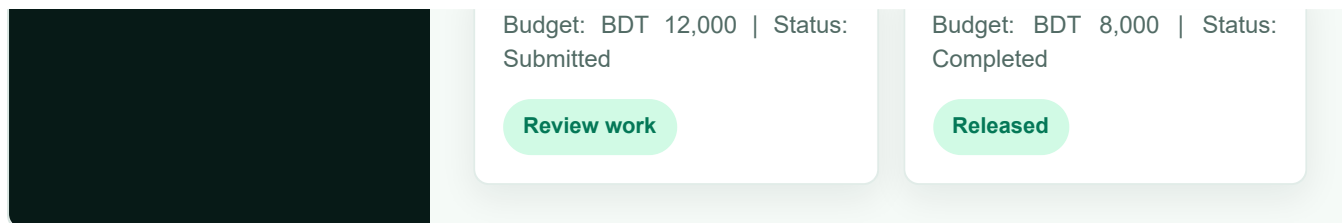


Fig-2.4.4: Client project management page

The Client Project Management Page allows clients to create projects, monitor status, view applicants, fund escrow, review submitted work, request revision, approve work, or open disputes when needed.

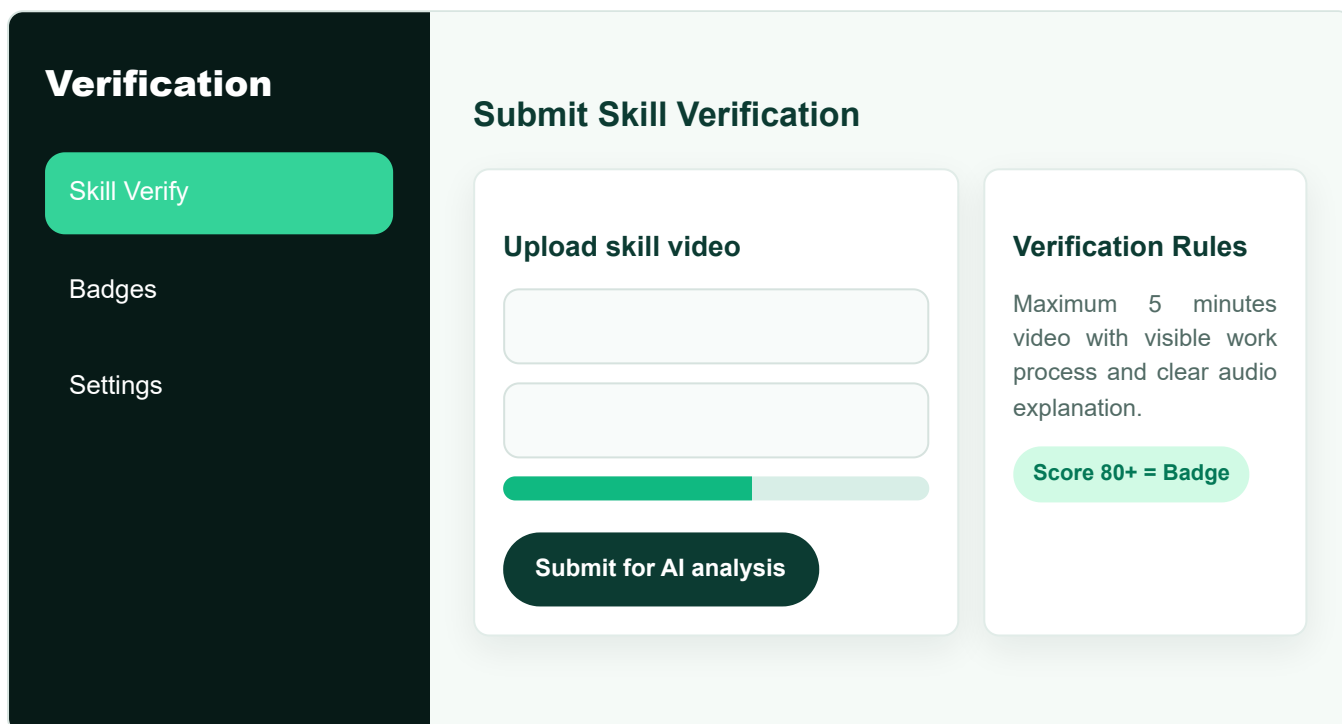
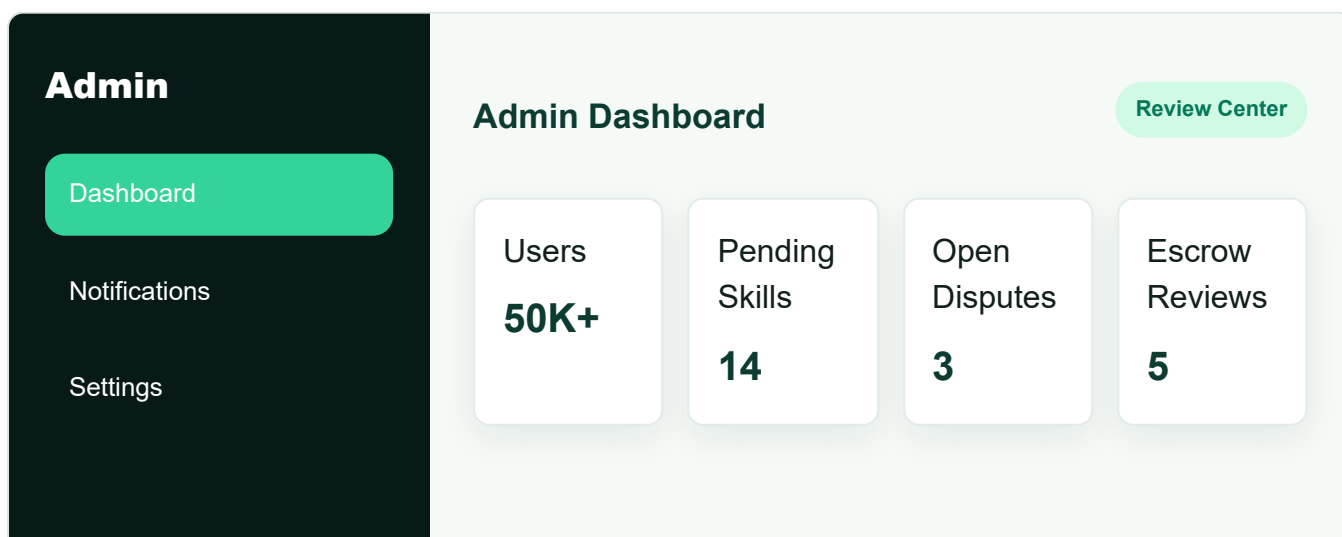


Fig-2.4.5: Skill verification page

The Skill Verification Page lets freelancers upload or submit short recorded work videos. The system validates the submission, starts AI analysis, and sends the result to administrators for final review.



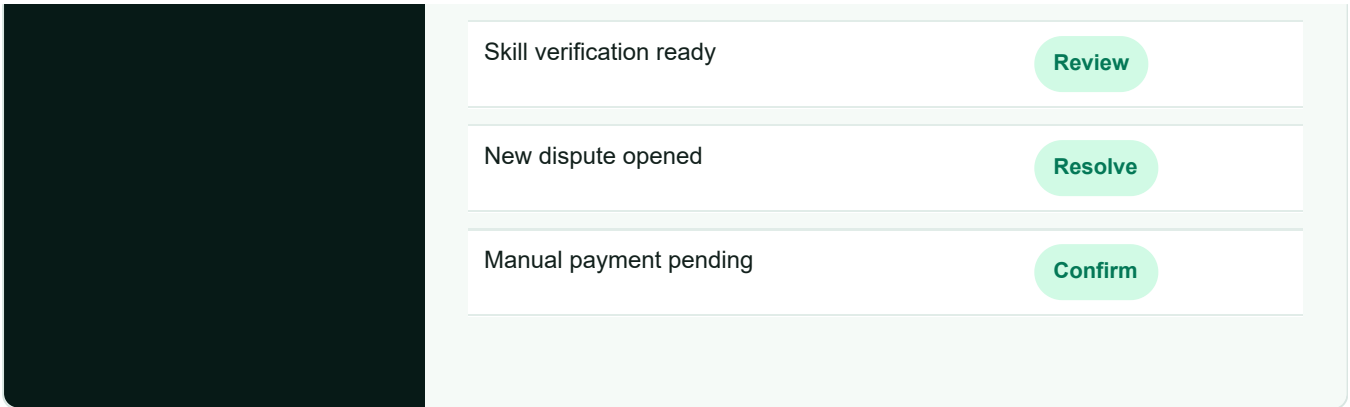


Fig-2.4.6: Admin dashboard and review center

The Admin Dashboard provides an overview of users, verification submissions, disputes, and payment review items. Administrators can review AI skill results, resolve disputes, and confirm manual escrow payments.

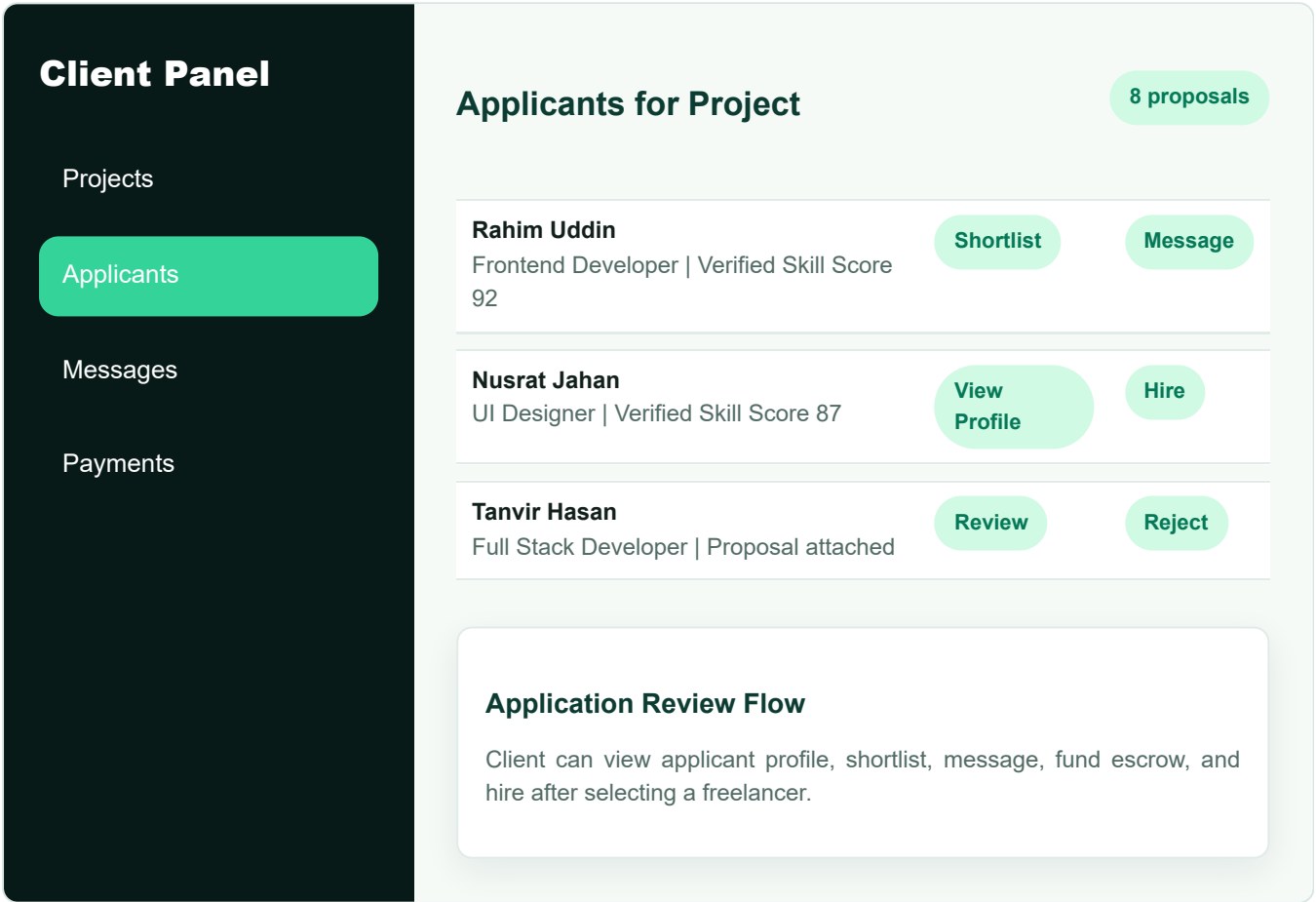
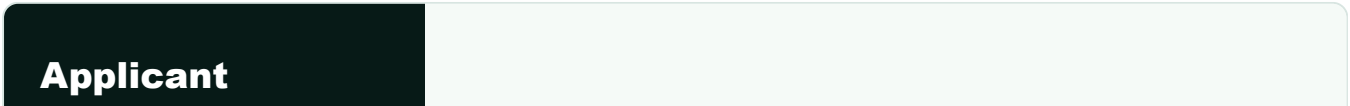


Fig-2.4.7: Applicants list page

The Applicants Page appears after a client opens the applicant list for a project. It shows freelancer name, profile summary, verified skill score, proposal details, and action buttons for shortlist, message, reject, or hire. This design follows the actual project flow where applicants are connected to project applications.



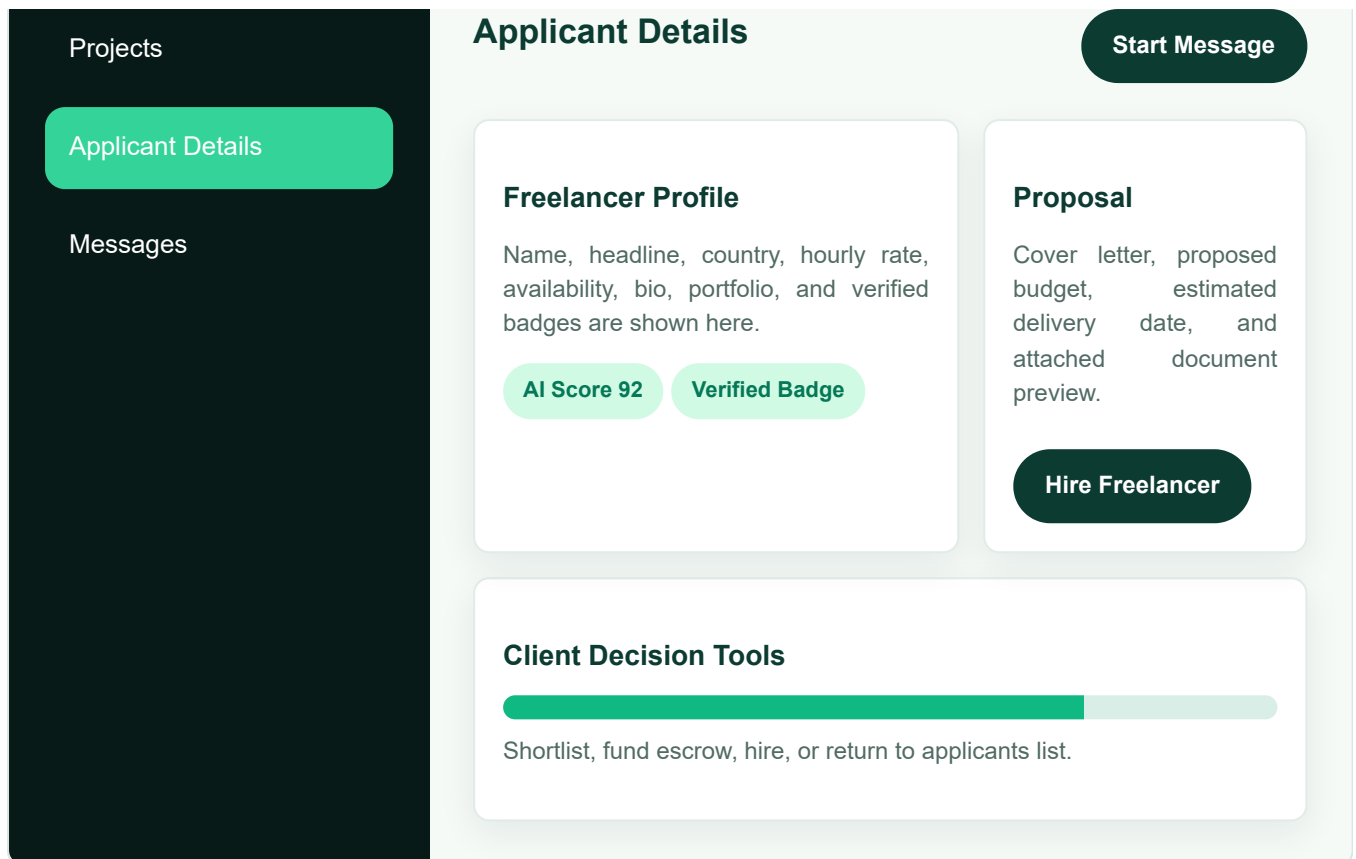
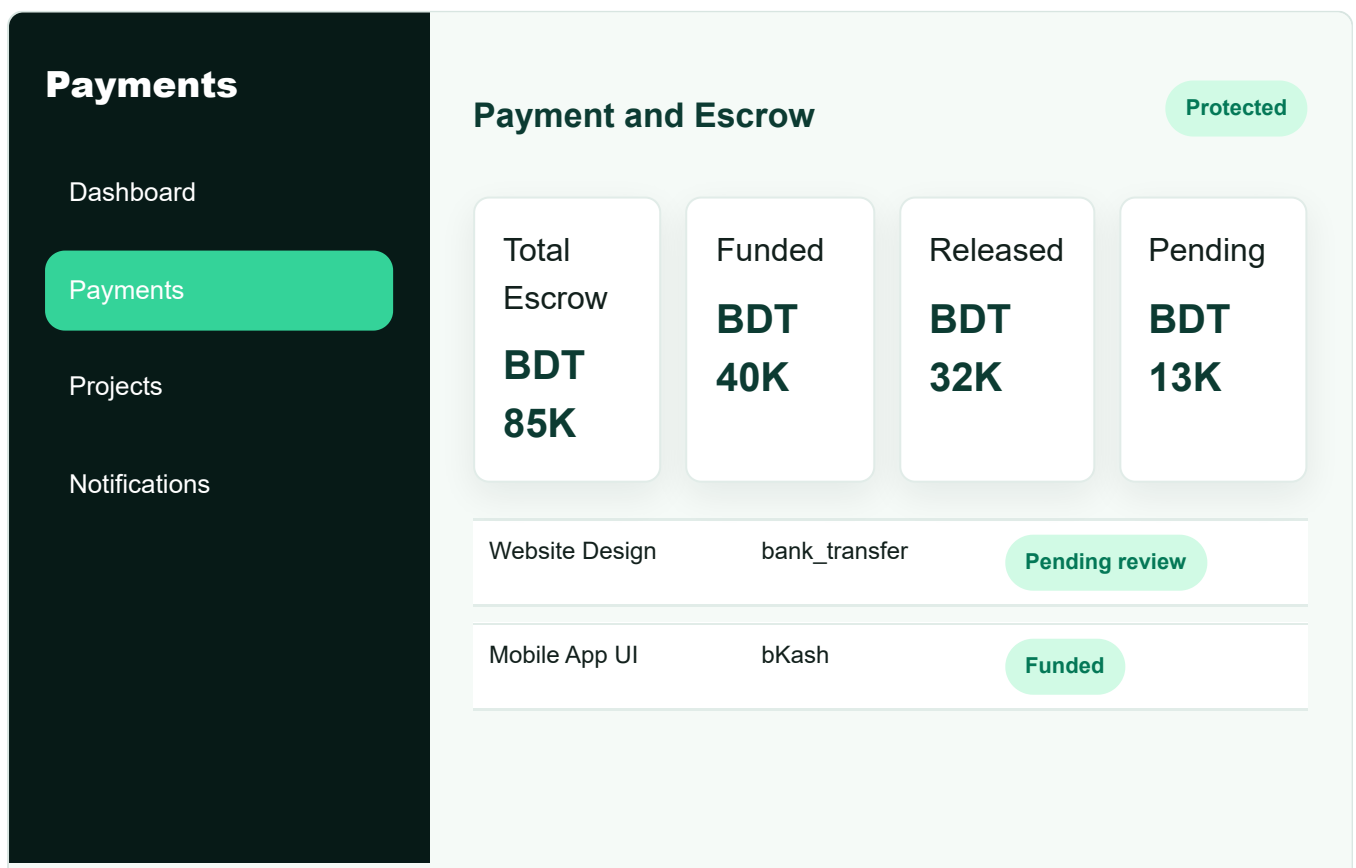


Fig-2.4.8: Applicant details page

The Applicant Details Page gives the client a detailed view of a freelancer before hiring. It contains profile information, skill badges, proposal text, budget, delivery date, and buttons for messaging, escrow funding, and hiring.



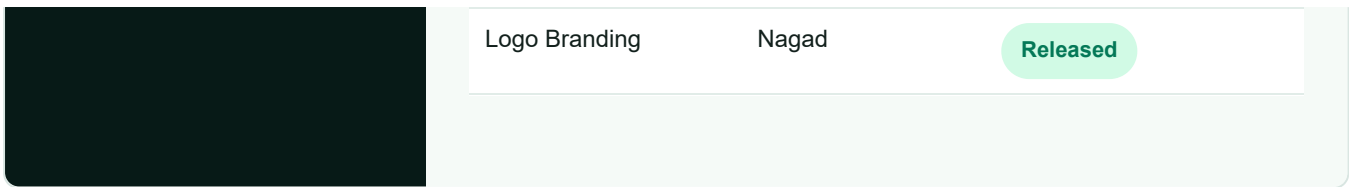


Fig-2.4.9: Payments and escrow page

The Payments Page shows escrow transactions, provider method, transaction reference, funded amount, released amount, and payment status. Clients use this section to fund escrow and freelancers use it to track released payments.

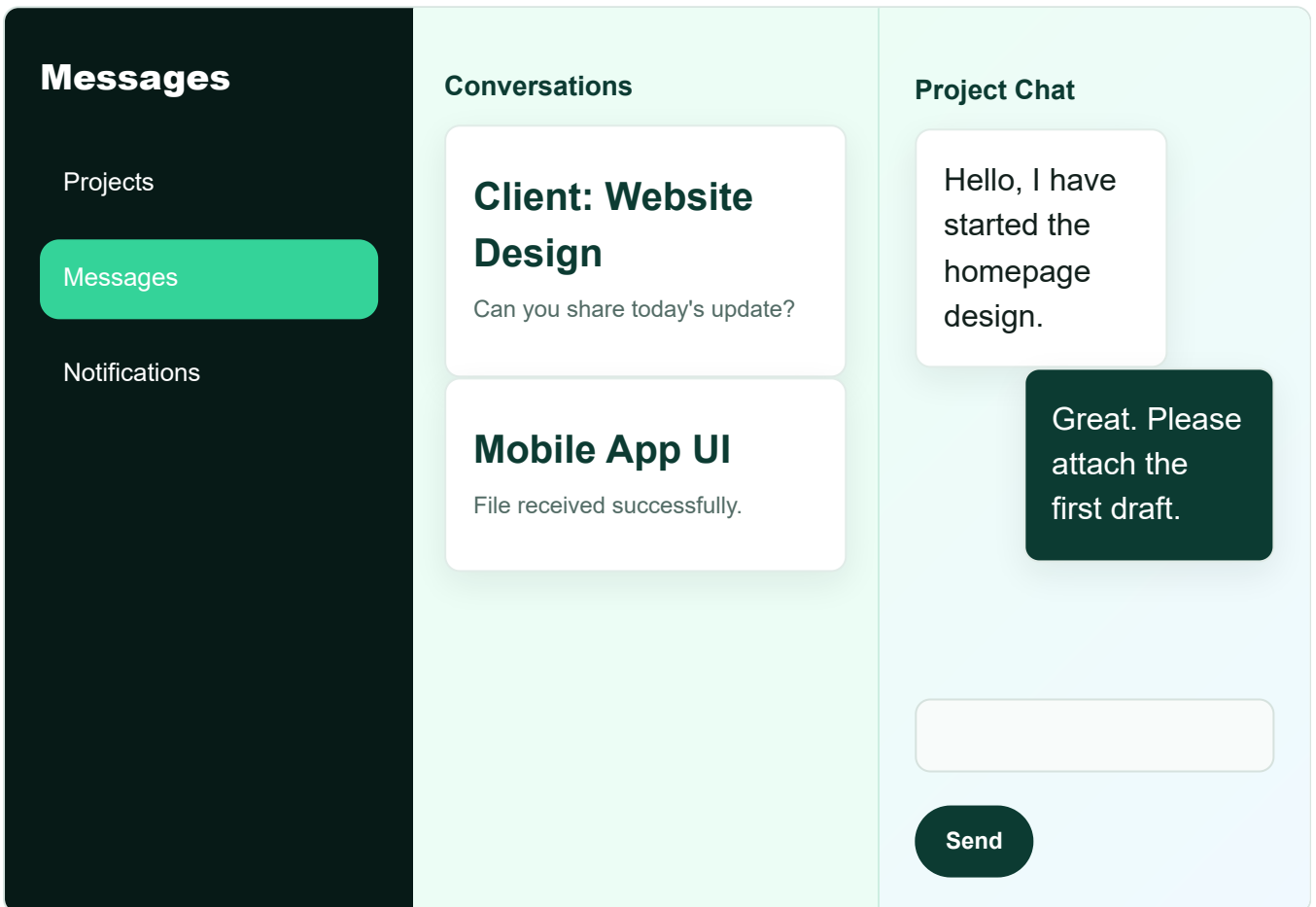
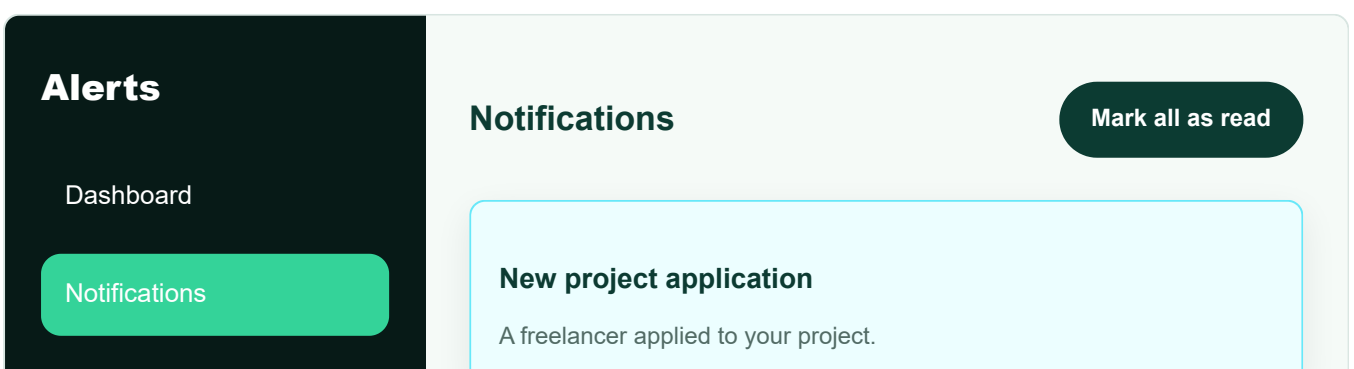


Fig-2.4.10: Real-time messages page

The Messages Page supports project based communication between client and freelancer. It includes a conversation list, chat area, attachment buttons, typing updates, message status, and send controls.



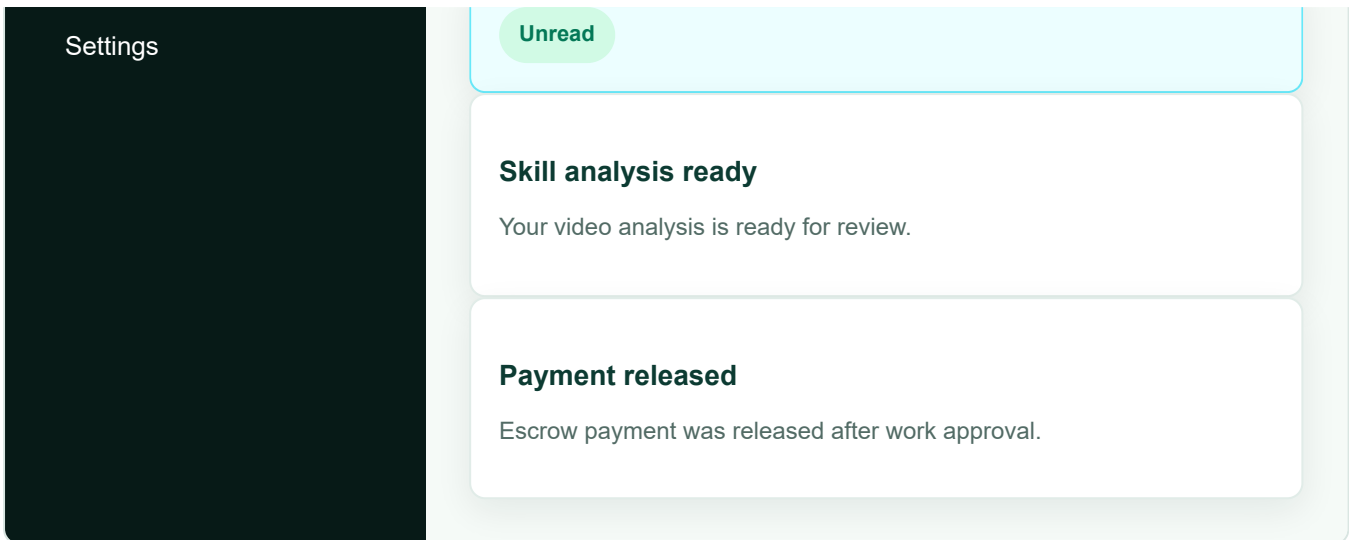


Fig-2.4.11: Notifications page

The Notifications Page lists project, payment, message, dispute, and skill verification updates. Unread notifications are highlighted. Clicking a notification routes the user to the related screen such as messages, projects, applicants, payments, or verification.

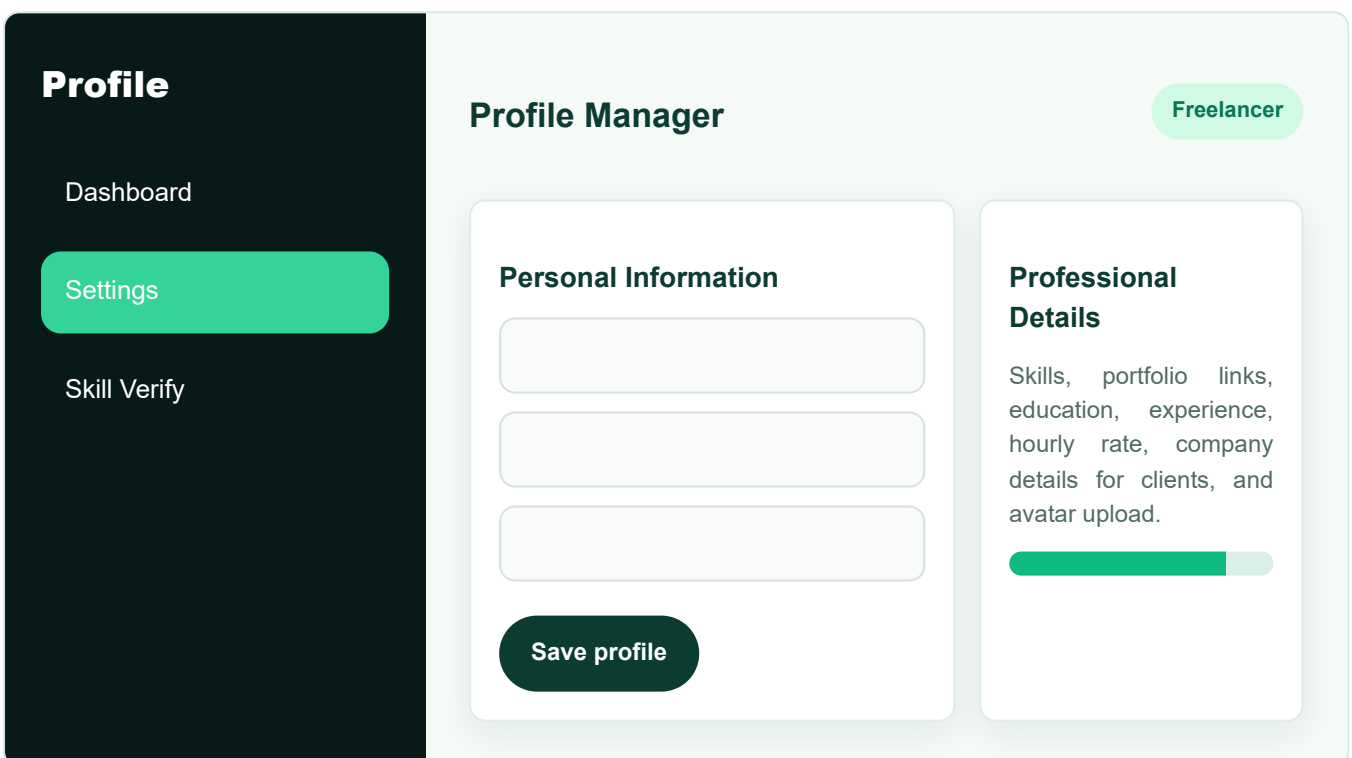
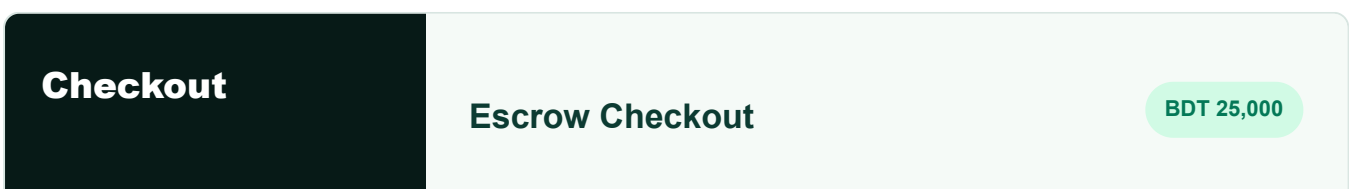


Fig-2.4.12: Profile and settings page

The Profile and Settings Page allows users to update personal and professional information. Freelancers can add skills, portfolio, experience, hourly rate, and availability. Clients can add company name, website, and business details.



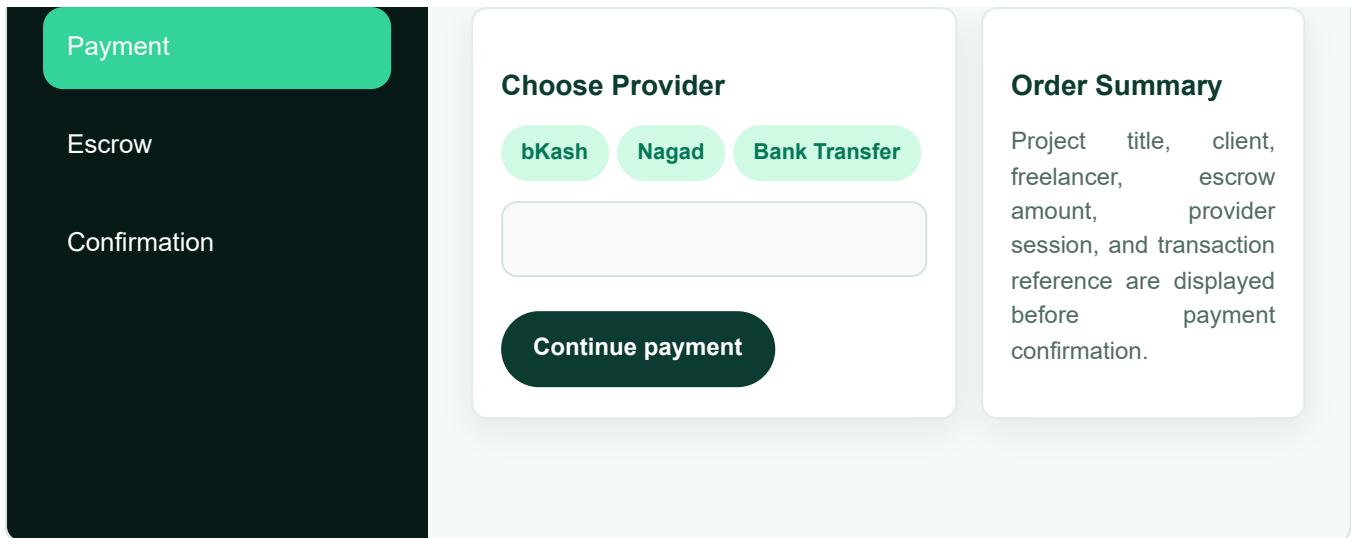


Fig-2.4.13: Checkout and escrow funding page

The Checkout Page is used when a client funds escrow. It displays selected project, amount, payment provider, transaction reference, and confirmation action. Manual payment providers require admin verification before the escrow becomes funded.

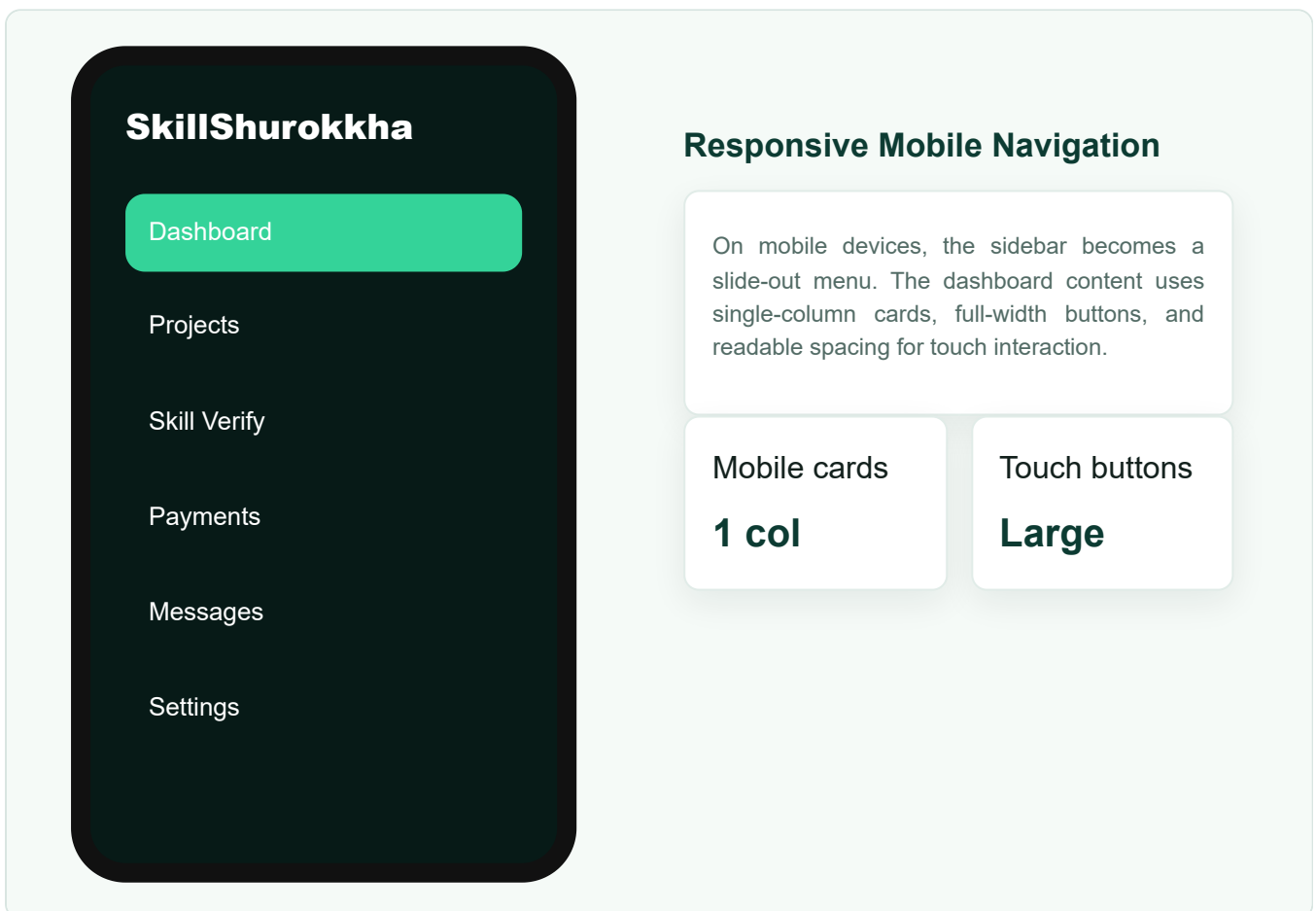


Fig-2.4.14: Mobile responsive sidebar and dashboard layout

The Mobile Responsive Layout shows how the dashboard navigation adapts to smaller devices. The sidebar opens as a menu, text remains readable, controls are large enough for touch, and dashboard sections stack vertically.

2.5 UML and System Design

2.5.1 Context Diagram

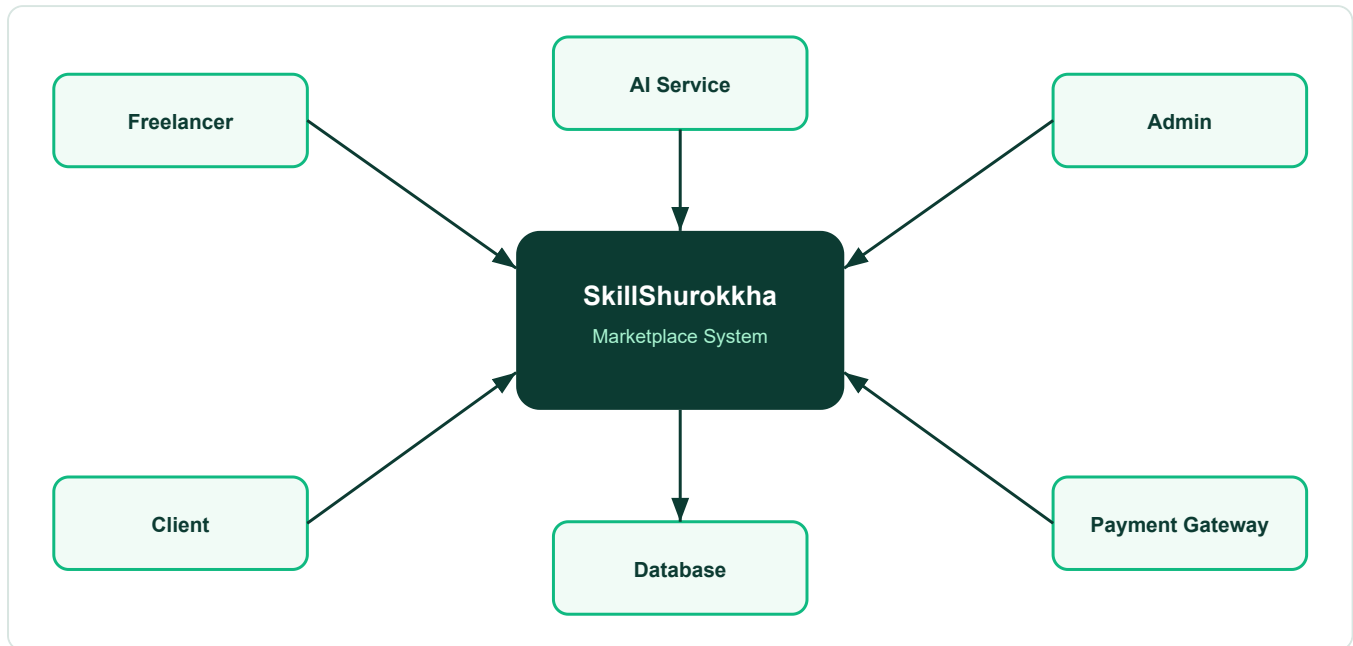


Fig-2.5.1: Context Diagram

The Context Diagram shows SkillShurokkha at the center. Freelancers submit profiles, applications, messages, and skill videos. Clients post projects, fund escrow, hire freelancers, and approve work. Administrators review users, disputes, payments, and skill verification. The system communicates with AI services, payment gateways, and the database.

2.5.2 Use Case Diagram

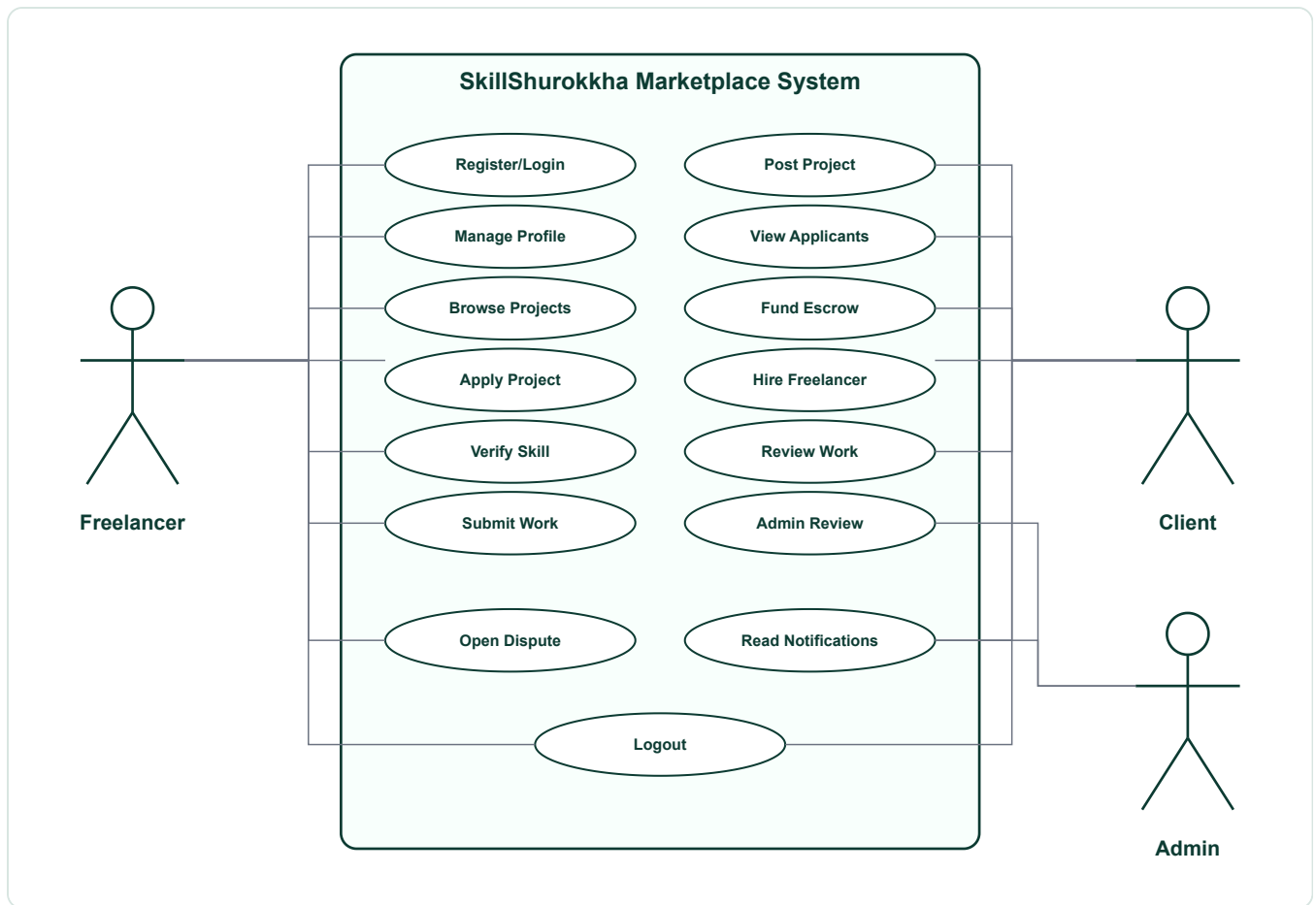


Fig-2.5.2: Use Case Diagram

The Use Case Diagram identifies the complete functional interactions among Freelancer, Client, and Admin. Freelancers register, manage profile, browse projects, apply, verify skills, submit work, message clients, receive payment, open disputes, and read notifications. Clients post projects, view applicants, shortlist, fund escrow, hire freelancers, review work, request revisions, approve payment, open disputes, and read notifications. Administrators manage users, review skill verification, confirm manual escrow, and resolve disputes.

2.5.3 Class Diagram

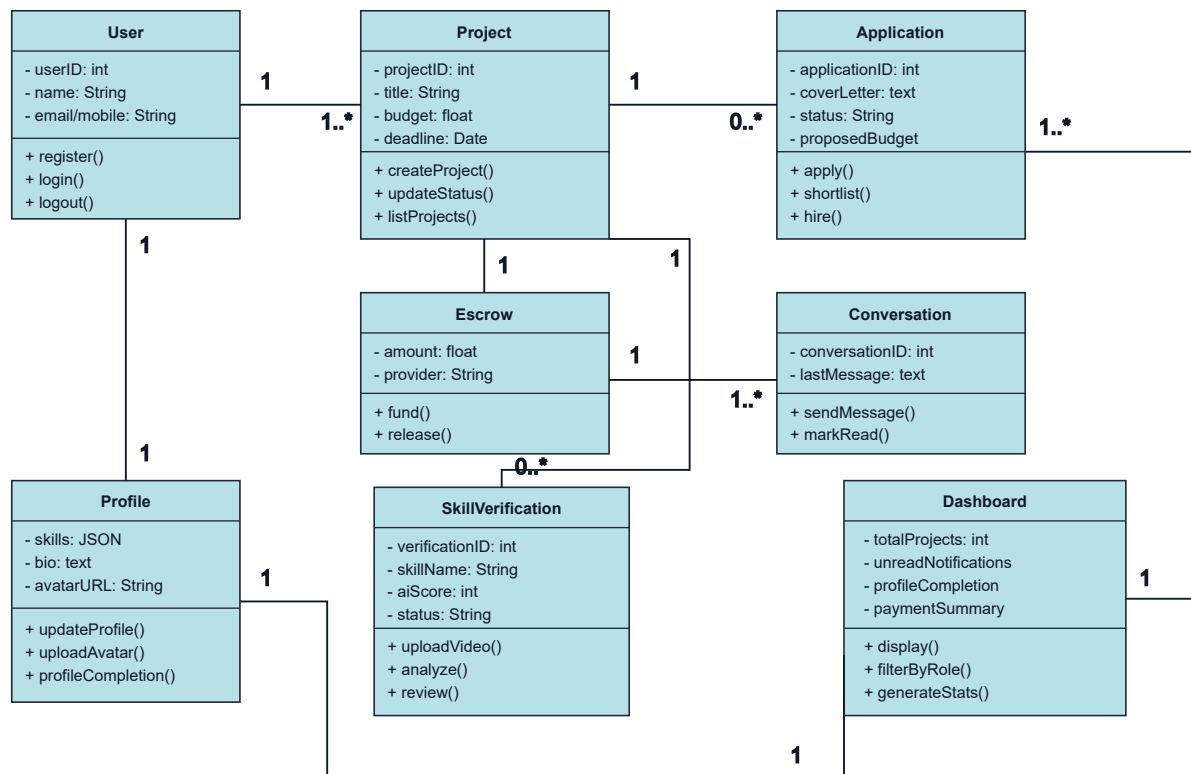


Fig-2.5.3: Class Diagram

The Class Diagram shows the object-oriented structure of SkillShurokkha with class attributes, methods, and multiplicity. A User owns one Profile and can create many Projects as a client or many Applications as a freelancer. Project connects to Application, Escrow, Conversation, and Dashboard data. SkillVerification stores AI score and review status for freelancer skills.

2.5.4 Activity Diagram

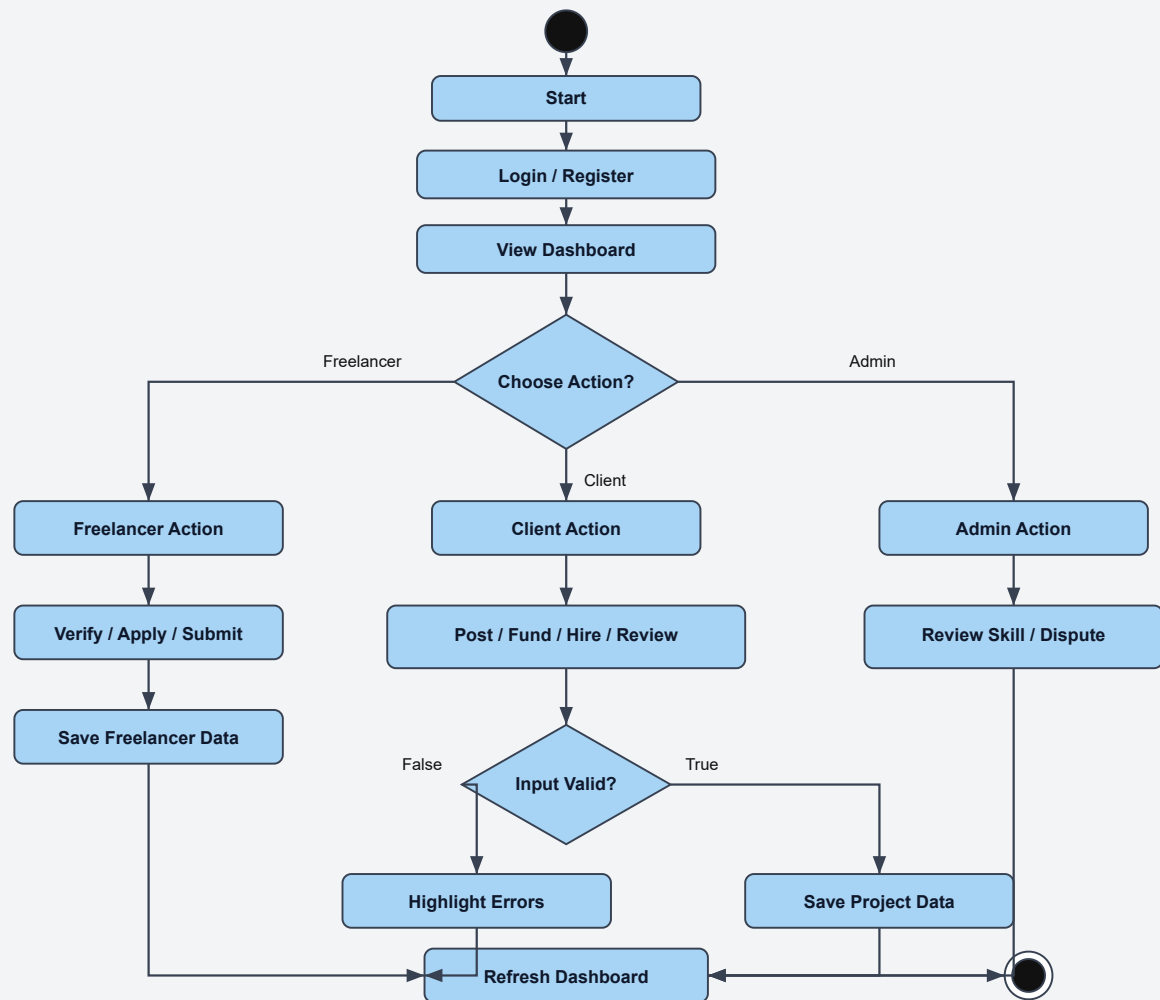


Fig-2.5.4: Activity Diagram

The Activity Diagram shows the complete role based workflow. After login, the user views the dashboard and chooses a role-specific action. Freelancers verify skills, apply to projects, or submit work. Clients post projects, fund escrow, hire freelancers, or review submissions. Administrators review skill results and disputes. Invalid input returns to error handling, while valid actions update project, escrow, notification, and dashboard data.

2.5.5 Sequence Diagram

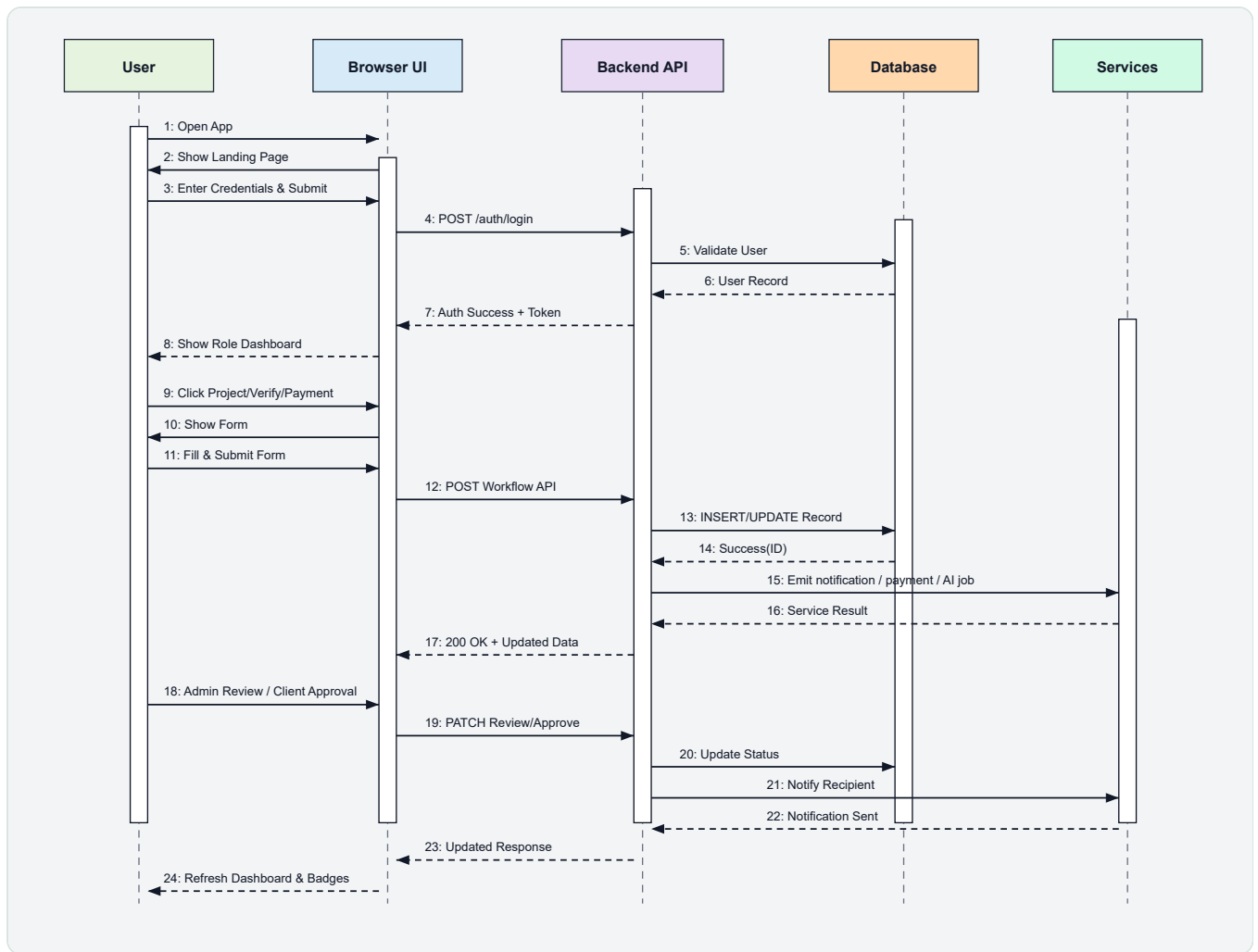


Fig-2.5.5: Sequence Diagram

The Sequence Diagram illustrates the end-to-end interaction flow. A user logs in through the browser UI, the backend validates credentials using the database, and the role based dashboard is displayed. For project, skill verification, payment, and review workflows, the browser sends API requests to the backend, records are updated in the database, external services such as AI, payment, and notifications are triggered, and the dashboard refreshes with updated data.

2.5.6 Entity Relationship Diagram

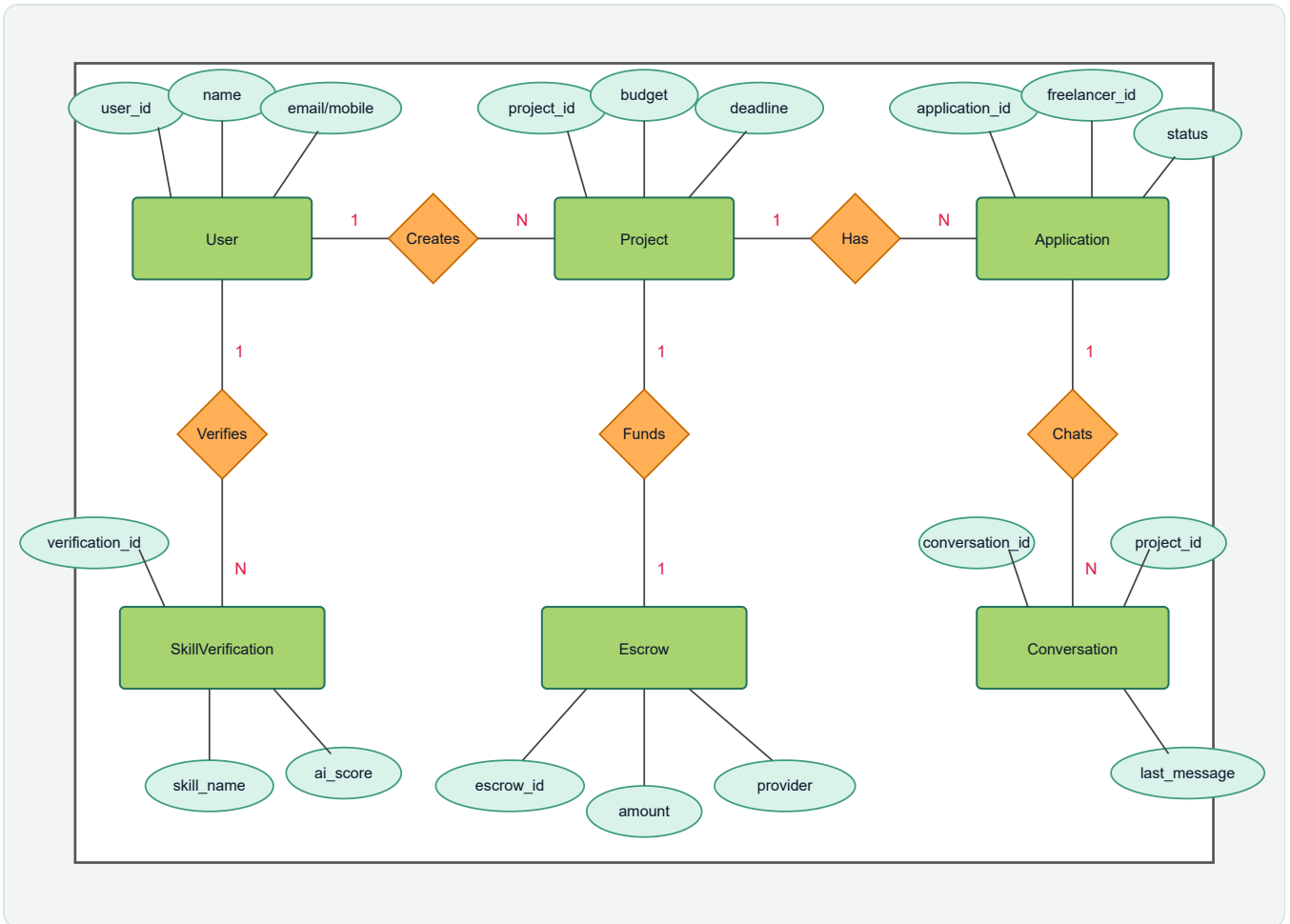


Fig-2.5.6: ER Diagram

The ER Diagram models the complete data structure of SkillShurokkha using entity boxes, relationship diamonds, attributes, and cardinality. User creates Project, submits Application, and verifies SkillVerification records. Project is connected with Application, Escrow, and Conversation. Escrow stores secured payment data, and Conversation stores project based messaging.

2.6 Constraints

- The system requires internet connectivity for most operations.
- Skill verification quality depends on video clarity, audio, and AI service availability.
- Payment release depends on escrow status and client approval.
- Manual payment providers require administrator confirmation before hiring can proceed.
- The system assumes one active account role per user.
- Only modern browsers are supported for the web interface.

3. External Interface Requirements

3.1 User Interface Requirements

The interface shall be simple, responsive, and role based. The left sidebar shall contain navigation items according to the logged in role. Admin users shall see dashboard, notifications, and settings. Clients shall see dashboard, projects, applicants, payments, messages, notifications, and settings. Freelancers shall see dashboard, projects, verification, payments, messages, notifications, and settings. Buttons, forms, badges, and alert messages shall be clear and consistent across pages.

3.2 Hardware Interface Requirements

- Desktop, laptop, tablet, and smartphone devices shall be supported.
- Camera and microphone shall be required for skill video recording or uploading.
- Stable internet connection through Wi-Fi, broadband, 3G, 4G, or 5G shall be required.
- No special hardware shall be required for normal project browsing and messaging.

3.3 Software Interface Requirements

Component	Requirement
Frontend	Next.js, React, Tailwind CSS, Socket.IO client
Backend	Node.js, Express.js, REST API, Socket.IO
Database	Relational database with users, projects, applications, escrows, messages, notifications, and skill verification tables
AI Service	Video metadata analysis, transcription, scoring, and skill authenticity review workflow
Payment	bKash, Nagad, bank transfer, manual review, or development gateway provider integration

3.4 Communication Interface Requirements

- The system shall use HTTPS in production for secure data transfer.
- The frontend shall communicate with backend REST APIs using JSON and multipart form data where needed.
- Real time messages and notification updates shall use WebSocket communication through Socket.IO.
- Payment gateways shall communicate through redirect callbacks and webhook endpoints.
- Users shall receive in-app notifications for applications, hiring, messages, payment release, disputes, and skill verification events.

4. Non-functional Requirements

4.1 Security

The system shall protect user accounts through secure authentication and authorization. Passwords shall be stored using hashing. Role based access control shall prevent unauthorized users from accessing admin, client, or freelancer actions. Payment and escrow workflows shall prevent direct payment release without proper approval. Sensitive data shall be transferred through HTTPS in production.

4.2 Capacity

The initial system shall support at least 50,000 users and allow future expansion. Video files, proposal documents, avatars, messages, and project records shall be stored in scalable storage. The database shall support indexing for notifications, projects, users, and conversations to keep common pages responsive.

4.3 Compatibility

The web application shall be compatible with modern browsers such as Chrome, Edge, Firefox, and Safari. The interface shall support desktop and mobile screen sizes. The system shall work on Windows, Linux, Android, and iOS through browser access.

4.4 Reliability

The system shall target 99% uptime under normal project usage. Important workflows such as escrow funding, hiring, dispute creation, and payment release shall use database transactions to reduce inconsistent states. User data shall not be lost after page refresh, logout, or temporary network interruption.

4.5 Scalability

The system shall be designed so backend services, database, file storage, and AI analysis can be scaled independently. Notification, messaging, and AI analysis processes shall support future workload growth. The architecture shall allow additional payment providers and verification models to be added later.

4.6 Maintainability

The system shall use modular backend routes and services. Project, payment, notification, message, and skill verification logic shall be separated into maintainable modules. Bug fixes and features shall be tested and deployed through a controlled development workflow.

4.7 Usability

The system shall be easy to use for non-technical freelancers and clients. Navigation shall be visible, actions shall use clear labels, and important statuses shall be shown through badges and notifications. Bangla and English language support shall improve accessibility for Bangladeshi users.

4.8 Other Non-functional Requirements

- The system shall respond quickly to common dashboard and project actions.
- Uploaded files shall be validated by type and size.
- Error messages shall be understandable and user friendly.
- The platform shall be prepared for local payment and data compliance requirements.

5. Definitions and Acronyms

Term	Definition
AI	Artificial Intelligence, used for skill video analysis and scoring.
API	Application Programming Interface, used for frontend and backend communication.
Client	A user who posts projects and hires freelancers.
Freelancer	A user who verifies skills, applies to projects, and completes work.
Admin	A platform user who reviews skill verification, disputes, payment, and user status.
Escrow	A secure payment holding process where client funds are locked until work approval.
OTP	One-Time Password used for account or contact verification.
UI	User Interface, the screens and controls users interact with.
UML	Unified Modeling Language, used to design system diagrams.
SRS	Software Requirement Specification, a document that describes software requirements.
Socket.IO	A real time communication library used for messages and notification updates.
Webhook	An endpoint called by an external payment provider to notify payment status.